

Sam Wisnoski

September 7th, 2026

Data Structures and Algorithms - Assignment 1

Instructors: Paul Ruvolo

(1) Choosing a Language & Course Entry Survey

C++, I choose you! As mentioned in the course entry survey, I want to code in C++ in order to hone my robotics skills, and I think it will be a good challenge. I also completed the survey.

(2) Identifying Effective Strategies for Learning

Respond to the following prompts to get you thinking about your approach to learning in this course.

[1] Choose a moment in your educational career where learning went really well. What strategies did you employ that worked particularly well?

It's tricky – I feel like the moments where “learning went really well”, I often attribute to great professors and interesting material rather than anything that I did differently. For example, I would consider my favorite and most successful classes Discrete Math, Machine Learning, and Computational Robotics, simply due to how interested I was in the material and the fact that the professors created an incredibly supportive classroom environment.

For the sake of this question, however, I'll focus on my Fundamentals of Robotics final project, where I designed a robot that would identify and pick up an unmarked block anywhere on a ArUco board (I choose this project because I was the primary coder, but most CompRobo projects would work here too). For this project, there were several new moving parts that I would need to interact with in order to successfully complete the project. Rather than jumping in from stage one and attempting to build the full system from start to finish, I instead broke the project into its respective components, then the components into their respective functions, and lastly, the functions into their respective inputs and outputs. Basically, I designed the system outline & pseudocode before I started coding.

It may seem like a simple and obvious choice, but this is Olin, where doing is often preferred to planning. For me, however, I learn better when I can understand each miniscule function and mathematical property of a system, and that requires complex planning from stage 1 of the project.

[2] Which sorts of strategies have led to either less effective learning or less enjoyment of the learning experience. Feel free to describe a few examples of what doesn't work for you.

It's rarely been a problem here at Olin, but it's a pet peeve of mine when assignments are designed to be completed in a specific format (such as Markdown/LaTeX/On Paper) unrelated to course material. Broadly speaking, the over-restriction of how assignments are completed leads to less creativity, less personalization, and less learning. I prefer when the start and end goals are specified, and we can fill in the blanks with our work as we go – although importantly, this is only relevant for things OUTSIDE of course material. In DSA, for example, I would expect we would need to follow specific good coding practices, but I would prefer we are given freedom in how we complete projects and write-ups.

Another thing that hasn't worked well for me in the past is widely differing week to week assignments. I've taken a few classes that ramp up significantly in the later half of the semester, a few that ramp up around midterms and then work fades after Thanksgiving, and a few where it periodically shifts from week to week. The best courses I've taken, however, assign work consistently, where we can (relatively) plan how long an assignment will take each week and make sure we set aside enough time for it. For a double negative, classes that have harsh punishments for late assignments but assign a varying amount of work from week to week make it incredibly difficult to get the most out of assignments.

Lastly, I've had a few negative experiences with forced teams. There are a few scenarios where it makes sense, but largely, I think we should be allotted the freedom to choose our teammates.

[3] As this course is foundational for many aspects of computer science, the problems in this class can be easily solved with modern AI systems. One of my foundational assumptions is the process of grappling with a problem helps you internalize the important concepts, gives you more insight into how the tools you are learning can be applied in other contexts, helps you more realistically assess your own abilities, and helps you learn to better communicate your knowledge to others. Particular methods of using AI are unlikely to achieve the learning goals articulated previously. Do you agree with this framing? How are you thinking about AI tools with respect to this course?

As an avid user of AI this summer, I do agree with this framing. I am on a personal quest this semester to use AI as little as possible (for coursework) this semester, since I think it severely deteriorates the learning experience in pretty much all aspects. While it *can* be useful as an evaluator or teacher, I think it's much more likely to become a crutch and a shortcut. Especially

with learning a new language, I think if I start heavily using AI on even one assignment, it will quickly make the rest of the course much more difficult without it.

However, I'm taking this course *purely* to learn, and because I think DSA is fun. I'm not interested in technical interviews, nor a career in software engineering. I think if I were a sophomore worried about how AI could affect my career trajectory, my answer to the above question would be very different.

[4] What strategies will you use in this course to be successful? With respect to AI, what principles or strategies will you use during this course.

As mentioned above, I'll try to break down larger problems into simple components that I can individually understand. Additionally, I'd like to be proactive in showing up to class, completing assignments, using good coding & documentation practices, and asking questions when I don't understand things. There's a lot of specifics I could drone on about here, but largely, I strive to be communicative, transparent, and honest in my work and learning.

With respect to AI, I will not be using AI in any meaningful capacity. I may occasionally throw a definition or googling question to AI, or have it synthesize a Wikipedia page for me, but I will not be using it to write any code or complete assignments for me in any fashion.

[5] What do you think of some of the proposed activities for the oral quizzes? Are these activities ones that would give you helpful feedback as to how you are performing with respect to the course material? Would you add or subtract any of the proposed activities?

Largely, I think the oral quizzes are a great idea, and the proposed activities sound *mostly* great. Admittedly, I haven't taken an oral quiz before, but I think providing rubric or additional expectations beforehand would help ease some tension and clarify what we are being tested and graded on. It also is slightly worrying that I may encounter an unfamiliar coding language as part of a quiz.

The activities themselves are great, they all seem to test both understanding and implementation of concepts rather than nitpicky specifics. I do have a specific preference of which ones I would prefer to engage with, but I don't think I would add or remove any. If, hypothetically, I was working through a piece of code with errors, it would be preferred that the errors were DSA related errors (not syntax or spelling), and that the code would be provided in an IDE/debugging friendly environment. Other than that, all good!

[6] How can the teaching team support you?

As mentioned in the course survey, I am potentially interested in a Pass/No Credit option for this course. I appreciate being responsive to email and extending deadlines if given reasonable advance notice, but no pressure to do so. And lastly, early feedback is always great! The more I hear back sooner, the more I can iterate, improve, and learn!

(3) Hello World!

Since I am using C++ as my language of choice, I followed these two resources to get C++ up and running in my local windows environment: [C/C++ for Visual Studio Code](#) and [Configure VS Code for Microsoft C++](#). I then walked through a few different pages on [C++ Tutorial](#) to get a basic grasp of the language. This, at the very least, was enough to get me running a program.

[1] What features do you like about C++?

I actually don't mind the static language part as much as I was expecting. It seems to give you a lot more control over the specific operations that you're performing, but I'm still getting used to the syntax. I've tried to learn C++ before for robotics & interviews so I have some familiarity, but it's still rough for sure.

[2] Are there things you were expecting to find that you haven't?

I always forget how annoying it is to work with a non-standard compiler. What do you mean there are two main windows compilers that are recommended? Why can't there just be one? Why can't it be inbuilt? Why didn't I just choose python for this class?

[3] What questions do you have?

No questions yet, although I'm curious to know who else is working in C++, so we could potentially learn and work together. I know Oscar and Vivian are, but unsure besides that.

[4] Try using the debugger for some very basic information on the debugger. Do you have experience using interactive debuggers like this one? Were you able to successfully launch the debugger?

I was able to successfully launch the C++ debugger. It seems like a standard debugger, similar to ones I've used before, but maybe I'll learn to love it in time.

(4) Translating Your Old Code

Choose a piece of code that you've written in a programming language other than C++.

Translate the code to run in C++. The code you produce should be documented and contain at least some unit tests.

Include a link to a GitHub repo containing your port of the code. Include a writeup that describes the purpose of the code and how you found the process of translating the code to C++. Feel free to touch on the good, the bad, and the ugly.

Github link: https://github.com/swisnoski/DSA_Assignment_1/tree/main

Freshman fall, I did a passionate pursuit in Python, as Java was the only language I knew at the time and I wanted to be more prepared as I began working on more complex projects. At the end of the passionate pursuit, I created a simple chess game, which I've now attempted to port over to C++.

I say attempted because it IS now functional in C++, but only loosely so. It doesn't contain any of the fanciness of the python version, but it does use SDL (which I learned is what pygame is built on!) to render a basic board and pieces. You can select a piece and move it to any other piece on the board, which is sorta how physical chess works. I talk a bit more about this in the comments of the code.

Over all, it ended up being a much bigger time sink than I had originally anticipated. It took FOREVER to get SDL up and running properly, and then just as long to try and fail to include a second library (SDL_image). However, after spending multiple hours translating code, I do feel much more comfortable with C++ that I did before I started, so even if the end product isn't perfect, I feel like I achieved the learning goal for this assignment. A lot of the remaining work would have been repetitive to what already exists in the codebase anyway.

(5) Implementing Meeting Scheduler

Implement an algorithm that determines whether a collection of meetings contains a conflict (as we discussed in the [day 1](#) page).

It's up to you how you pass data into your program. You could read it from a file, hard code a test input into your main function, etc.

- **First implement the straightforward algorithm that checks pairs of meetings.**
- **Develop a second algorithm that sorts the meetings in a useful way so you can check for conflicts more easily. You should not implement your own search function, but instead use your language's built-in sort function.**
- **Write unit tests for both implementations, including edge cases such as one meeting ending exactly when another starts. For each algorithm (the straightforward one and the one based sorting), describe how you expect its running time to grow with n . As mentioned on day 1, sorting has runtime $\Theta(n \log n)$.**

And finally, Meeting Scheduler. After too many hours of chess, this algorithm felt like a breeze. I was able to apply a few tricks using regular expressions to make it easy to filter data, leading my time extractions to simply be comparisons between substrings. The exact strategies are outlined in the code, and the full writeup to this problem was completed in class, so I won't spend too much time on it here other than to answer the final part of the last question.

For algorithm one, the "straightforward" algorithm, it's running time would grow at a rate of n^2 . This is because we pair each meeting with each other.

For algorithm two, the "sorting" algorithm, we can expect it's running time to grow at a rate of $n \log(n) + n$, or just $n \log(n)$ since it relies on sorting the values first ($n \log(n)$) before going through each value one at a time.